

Paralelização de Algoritmos Clássicos com OpenMP:

Análise de Desempenho, Speedup e Eficiência

Lucas Pontes Lira* Caio Sena Freitas†

Centro Federal de Educação Tecnológica Celso Suckow da Fonseca (CEFET-RJ)

UNED Nova Friburgo — Bacharelado em Sistemas de Informação

Abstract

Processamento paralelo virou uma das formas mais diretas de extrair desempenho de processadores modernos com vários núcleos. Nesse trabalho a gente paraleliza quatro algoritmos clássicos — multiplicação de matrizes, Selection Sort, Heap Sort e histograma — usando OpenMP em C. Para cada um foram feitas uma versão serial de referência e uma versão paralela, com análise das dependências de dados e das regiões críticas. Os testes rodaram variando o tamanho das entradas e o número de threads (1, 2 e 4), medindo tempo de execução, speedup e eficiência. Multiplicação matricial e histograma escalaram bem; os dois sorts tiveram ganhos bem menores por causa das dependências sequenciais que não dá pra fugir. A comparação entre Selection Sort e Heap Sort mostrou diferenças no comportamento de cache e no balanceamento de carga que explicam boa parte das diferenças no speedup.

Palavras-chave: OpenMP, computação paralela, multiplicação matricial, algoritmos de ordenação, histograma, speedup, eficiência.

Abstract

Parallel processing has become one of the most straightforward ways to extract performance from modern multi-core processors. In this work we parallelize four classical algorithms — matrix multiplication, Selection Sort, Heap Sort, and histogram computation — using OpenMP in C. For each algorithm we developed a serial reference implementation and a parallel version, analyzing data dependencies and critical regions. Experiments ran with varying input sizes and thread counts (1, 2, and 4), measuring execution time, speedup, and efficiency. Matrix multiplication and histogram scaled well; the sorting algorithms showed smaller gains due to inherent sequential dependencies. The comparison between Selection Sort and Heap Sort revealed differences in cache behavior and load balancing that explain much of the speedup gap.

Keywords: OpenMP, parallel computing, matrix multiplication, sorting algorithms, histogram, speedup, efficiency.

*lucas.lira@aluno.cefet-rj.br

†caio.freitas@aluno.cefet-rj.br

Contents

1	Introdução	3
2	Fundamentação Teórica	3
2.1	Modelos de Computação Paralela	3
2.2	OpenMP: Modelo de Execução	3
2.3	Métricas de Desempenho	4
2.4	Lei de Amdahl	4
2.5	Lei de Gustafson	4
2.6	Condição de Corrida e Regiões Críticas	5
2.7	Dependência de Dados	5
2.8	Cache e False Sharing	5
3	Trabalhos Relacionados	5
4	Metodologia	6
4.1	Ambiente de Execução	6
4.2	Organização dos Experimentos	6
4.3	Medição de Tempo	6
4.4	Estrutura dos Arquivos	6
4.5	Verificação de Correção	6
5	Desenvolvimento	7
5.1	Multiplicação Matricial	7
5.1.1	Descrição	7
5.1.2	Complexidade	7
5.1.3	Implementação Serial	7
5.1.4	Dependências de Dados	8
5.1.5	Implementação Paralela	8
5.1.6	Resultados	9
5.2	Selection Sort	9
5.2.1	Descrição	9
5.2.2	Complexidade	10
5.2.3	Implementação Serial	10
5.2.4	Dependências de Dados	10
5.2.5	Implementação Paralela	11
5.2.6	Resultados	12
5.3	Heap Sort	12
5.3.1	Descrição	12
5.3.2	Complexidade	13
5.3.3	Implementação Serial	13
5.3.4	Dependências de Dados	14
5.3.5	Implementação Paralela	14
5.3.6	Resultados	15
5.4	Histograma	16
5.4.1	Descrição	16
5.4.2	Complexidade	16
5.4.3	Implementação Serial	16

5.4.4	Condição de Corrida	17
5.4.5	Implementação Paralela	17
5.4.6	Resultados	19
5.5	Makefile	19
6	Resultados e Discussão	20
6.1	Visão Geral do Speedup	20
6.2	Speedup em Função do Tamanho	21
6.3	Comparação Selection Sort vs Heap Sort	22
6.3.1	Qual gera mais cache-misses?	22
6.3.2	Qual se beneficiou mais do paralelismo?	22
6.3.3	Balanceamento de carga	23
6.4	Análise pela Lei de Amdahl	23
7	Limitações	23
8	Considerações Finais	24

1 Introdução

Quando começamos a estudar computação paralela a pergunta mais óbvia é: “se o processador tem 6 núcleos, por que o programa não roda 6 vezes mais rápido?”. A resposta não é simples, e explorar ela na prática foi um dos principais motivadores desse trabalho.

Processadores com múltiplos núcleos são o padrão hoje em dia, tanto em máquinas pessoais quanto em servidores. O problema é que um programa escrito de forma convencional usa só um núcleo. Para aproveitar os outros é preciso dividir o trabalho explicitamente, o que exige cuidado porque nem todo algoritmo divide bem.

O OpenMP é uma das ferramentas mais usadas pra isso em sistemas de memória compartilhada. Ele funciona basicamente adicionando diretivas de compilador (`#pragma omp ...`) no código C existente, sem precisar reescrever tudo do zero. Pra fins acadêmicos funciona bem porque o código ainda é legível.

Escolhemos quatro algoritmos justamente porque eles têm características bem diferentes quanto à paralelizabilidade. A multiplicação de matrizes é quase um caso ideal: os cálculos de cada elemento da matriz resultado são independentes entre si. O histograma tem um problema clássico de condição de corrida que precisa ser resolvido com cuidado. Os sorts (Selection e Heap) têm dependências de dados que limitam o quanto dá pra paralelizar, e comparar os dois revela coisas interessantes sobre cache e balanceamento de carga.

O artigo está organizado assim: a Seção 2 traz a base teórica de paralelismo e OpenMP. A Seção 3 comenta trabalhos relacionados. A Seção 4 descreve o ambiente de teste e a metodologia. A Seção 5 apresenta o desenvolvimento de cada algoritmo. A Seção 6 discute os resultados. A Seção 7 lista as limitações. A Seção 8 fecha com as considerações finais.

2 Fundamentação Teórica

2.1 Modelos de Computação Paralela

A taxonomia de Flynn [4] classifica os computadores em quatro categorias: SISD, SIMD, MISD e MIMD. Processadores multicore modernos se encaixam na categoria MIMD, onde vários fluxos de instrução operam sobre vários fluxos de dados ao mesmo tempo.

Dentro do MIMD tem dois modelos principais: memória distribuída, onde cada processador tem sua própria memória e a comunicação é por troca de mensagens (como no MPI); e memória compartilhada, onde todos os processadores acessam o mesmo espaço de endereçamento. O OpenMP funciona nesse segundo modelo.

2.2 OpenMP: Modelo de Execução

O OpenMP foi padronizado em 1997 e está na versão 5.2 desde 2021 [3]. Ele define um conjunto de diretivas para C, C++ e Fortran.

O modelo de execução segue o paradigma fork-join: começa com uma thread mestre; quando encontra um `#pragma omp parallel`, essa thread cria um time de threads auxiliares; ao final da região paralela todas sincronizam e só a mestre continua.

As diretivas mais usadas nesse trabalho:

- `#pragma omp parallel for` — distribui as iterações de um laço entre as threads.

- `#pragma omp parallel for reduction` — mesmo que o anterior mas combinando resultados parciais com segurança.
- `#pragma omp critical` — garante que só uma thread execute aquele bloco por vez.
- `#pragma omp atomic` — protege uma operação read-modify-write de forma mais eficiente que `critical`.
- Cláusulas `schedule(static/dynamic/guided)` — controlam como as iterações são distribuídas entre threads.
- Cláusulas `private, shared, firstprivate` — definem o escopo das variáveis.

2.3 Métricas de Desempenho

As três métricas principais são:

Tempo de execução (T): medido com `omp_get_wtime()`, que retorna o wall-clock time em segundos.

Speedup (S): razão entre o tempo serial e o paralelo com p threads:

$$S(p) = \frac{T_s}{T_p} \quad (1)$$

O speedup ideal seria $S(p) = p$, o que raramente acontece na prática.

Eficiência (E): fração do speedup teórico que realmente se aproveitou:

$$E(p) = \frac{S(p)}{p} = \frac{T_s}{p \cdot T_p} \quad (2)$$

2.4 Lei de Amdahl

A Lei de Amdahl [1] define o limite teórico do speedup dado que uma fração f do programa é serial:

$$S_{max}(p) = \frac{1}{f + \frac{1-f}{p}} \quad (3)$$

Quando $p \rightarrow \infty$, o speedup máximo converge pra $1/f$. Isso significa que mesmo 10% de código serial já limita o speedup máximo a $10\times$, não importa quantos processadores você tenha. Essa lei se mostrou bastante precisa nos nossos resultados.

2.5 Lei de Gustafson

A Lei de Gustafson [2] propõe uma perspectiva diferente: se o tamanho do problema cresce junto com o número de processadores, o speedup escalado é:

$$S_G(p) = p - f(p - 1) \quad (4)$$

A ideia é que problemas maiores tendem a ter frações seriais menores em proporção. Isso ficou bem visível nos nossos experimentos, principalmente na multiplicação matricial.

2.6 Condição de Corrida e Regiões Críticas

Uma condição de corrida acontece quando duas ou mais threads acessam uma variável compartilhada e pelo menos uma delas escreve. O resultado fica não-determinístico. A solução é sincronização: regiões críticas, atômicas ou reduções.

O custo de uma região crítica é proporcional a quantas vezes as threads tentam entrar nela. Se mal usada, pode jogar fora todo o ganho do paralelismo.

2.7 Dependência de Dados

Quando o resultado da iteração i de um laço é lido pela iteração $i + k$ (dependência RAW — Read After Write), as iterações não podem rodar em paralelo sem cuidado especial. Identificar essas dependências é o passo mais importante antes de paralelizar qualquer coisa.

2.8 Cache e False Sharing

Algoritmos que acessam endereços consecutivos de memória (localidade espacial) rodam mais rápido por aproveitar melhor o cache. Em sistemas multicore existe ainda o problema de *false sharing*: duas threads escrevendo em variáveis diferentes mas na mesma linha de cache invalidam o cache uma da outra, degradando o desempenho mesmo sem conflito real.

3 Trabalhos Relacionados

Dagum e Menon [5] foram dos primeiros a documentar o OpenMP de forma ampla, destacando o modelo incremental de paralelização de laços. Chapman, Jost e van der Pas [6] consolidaram as boas práticas em um livro que ainda é uma das referências mais citadas da área.

Na parte de multiplicação de matrizes, Goto e van de Geijn [7] mostraram como organizar os acessos à memória tem impacto enorme no desempenho. Anderson et al. [8] investigaram a distribuição de trabalho em operações de álgebra linear densa e chegaram a conclusão de que paralelizar o laço externo é a estratégia mais eficiente, o que condiz com o que a gente implementou.

Paralelizar sorts não é trivial. Singler, Sanders e Putze [10] analisaram a paralelização de algoritmos de ordenação da STL, mostrando que Merge Sort e Quick Sort têm estrutura recursiva que favorece o paralelismo com tasks. O Selection Sort é frequentemente citado como problemático por causa da dependência no laço externo. O Heap Sort fica no meio: a construção do heap dá pra paralelizar, mas a extração não.

Nakashima et al. [11] compararam OpenMP e MPI e concluíram que para problemas de memória compartilhada o OpenMP é mais fácil de programar. Para sistemas distribuídos ou quando a memória de um nó não é suficiente, MPI é necessário.

4 Metodologia

4.1 Ambiente de Execução

Os experimentos rodaram em um computador pessoal com Windows 10, processador AMD Ryzen 5 3500X (6 núcleos físicos, 6 threads lógicas). O compilador foi o GCC via MinGW-w64, com as flags `-fopenmp` e `-O2`. A flag `-O2` foi escolhida pra ter otimizações razoáveis sem esconder o comportamento paralelo.

4.2 Organização dos Experimentos

Para cada algoritmo foram usados três tamanhos de entrada, aproximadamente dobrando a cada incremento:

Table 1: Tamanhos de entrada utilizados

Algoritmo	Tam. 1	Tam. 2	Tam. 3
Multiplicação Matricial	350×350	700×700	1400×1400
Selection Sort	33.000	66.000	106.000
Heap Sort	20.000.000	40.000.000	80.000.000
Histograma	150.000.000	300.000.000	600.000.000

Para cada combinação de tamanho e número de threads (1, 2 e 4), o experimento rodou cinco vezes e o valor reportado é a mediana. Isso ajuda a reduzir o efeito de picos de uso do sistema operacional.

4.3 Medição de Tempo

O tempo foi medido só durante a fase de processamento, excluindo leitura e escrita de arquivos. Usamos `omp_get_wtime()`, que dá o wall-clock time com precisão de ponto flutuante dupla. Isso é melhor que `clock()` do C padrão porque conta o tempo real, não só o tempo de CPU da thread principal.

4.4 Estrutura dos Arquivos

Dois arquivos por algoritmo: sufixo `_serial` para a versão sequencial e `_paralelo` para a versão com OpenMP. Um Makefile único compila tudo.

4.5 Verificação de Correção

Antes de medir desempenho a gente verificou que as versões paralelas produzem os mesmos resultados que as seriais. Para os sorts foi comparação elemento a elemento; para o histograma foi comparação das frequências. Só depois de confirmar isso é que passamos para os experimentos.

5 Desenvolvimento

5.1 Multiplicação Matricial

5.1.1 Descrição

Dadas duas matrizes A ($m \times k$) e B ($k \times n$), o produto $C = A \times B$ tem dimensão $m \times n$ com elementos:

$$C[i][j] = \sum_{l=0}^{k-1} A[i][l] \cdot B[l][j] \quad (5)$$

Para matrizes quadradas de ordem n , isso dá $O(n^3)$ operações.

5.1.2 Complexidade

$O(n^3)$ no tempo, $O(1)$ de espaço auxiliar (excluindo as matrizes). Algoritmos como Strassen chegam a $O(n^{2.807})$ [9], mas o padrão cúbico foi usado aqui por ser o mais comum na prática.

5.1.3 Implementação Serial

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4 #include <libppc.h>
5
6 int main(int argc, char *argv[]) {
7     if (argc < 4) {
8         fprintf(stderr, "Uso: %s <ordem> <arq1> <arq2>\n", argv[0]);
9         return 1;
10    }
11    int n = atoi(argv[1]);
12
13    double *A = load_double_matrix(argv[2], n, n);
14    double *B = load_double_matrix(argv[3], n, n);
15    double *C = (double *)calloc(n * n, sizeof(double));
16
17    double t_inicio = omp_get_wtime();
18
19    for (int i = 0; i < n; i++) {
20        for (int j = 0; j < n; j++) {
21            double soma = 0.0;
22            for (int k = 0; k < n; k++)
23                soma += M(i, k, n, A) * M(k, j, n, B);
24            M(i, j, n, C) = soma;
25        }
26    }
27
28    double t_fim = omp_get_wtime();
29    printf("Tempo serial: %.6f s\n", t_fim - t_inicio);
30
31    save_double_matrix(C, n, n, "matriz_mult.out");
32    free(A); free(B); free(C);
33    return 0;
```


34 }

Listing 1: Multiplicação matricial serial

5.1.4 Dependências de Dados

O laço externo (i) não tem dependência entre iterações: calcular $C[i][j]$ é completamente independente de $C[i'][j']$ para $(i, j) \neq (i', j')$. A variável `soma` é local a cada par (i, j) . Isso torna o laço externo um candidato natural à paralelização, sem precisar de região crítica.

5.1.5 Implementação Paralela

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4 #include <libppc.h>
5
6 int main(int argc, char *argv[]) {
7     if (argc < 4) {
8         fprintf(stderr, "Uso: %s <ordem> <arq1> <arq2>\n", argv[0]);
9         return 1;
10    }
11    int n = atoi(argv[1]);
12
13    double *A = load_double_matrix(argv[2], n, n);
14    double *B = load_double_matrix(argv[3], n, n);
15    double *C = (double *)calloc(n * n, sizeof(double));
16
17    double t_inicio = omp_get_wtime();
18
19    /* cada thread pega um subconjunto de linhas de C */
20    #pragma omp parallel for schedule(static)
21    for (int i = 0; i < n; i++) {
22        for (int j = 0; j < n; j++) {
23            double soma = 0.0;
24            for (int k = 0; k < n; k++)
25                soma += M(i, k, n, A) * M(k, j, n, B);
26            M(i, j, n, C) = soma;
27        }
28    }
29
30    double t_fim = omp_get_wtime();
31    printf("Tempo paralelo (%d threads): %.6f s\n",
32          omp_get_max_threads(), t_fim - t_inicio);
33
34    save_double_matrix(C, n, n, "matriz_mult.out");
35    free(A); free(B); free(C);
36    return 0;
37 }

```

Listing 2: Multiplicação matricial paralela

O `schedule(static)` divide as n linhas igualmente entre as threads. Como cada linha dá o mesmo trabalho (n^2 operações), o balanceamento é perfeito e não faz sentido usar escalonamento dinâmico aqui.

5.1.6 Resultados

Table 2: Tempo de execução (s) — Multiplicação Matricial

Tamanho	1 thread	2 threads	4 threads
350×350	0,423	0,215	0,108
700×700	3,388	1,711	0,864
1400×1400	27,102	13,619	6,914

Table 3: Speedup — Multiplicação Matricial

Tamanho	2 threads	4 threads
350×350	1,97	3,91
700×700	1,98	3,92
1400×1400	1,99	3,92

Table 4: Eficiência — Multiplicação Matricial

Tamanho	2 threads	4 threads
350×350	0,985	0,978
700×700	0,990	0,980
1400×1400	0,995	0,980

A multiplicação matricial foi o algoritmo que melhor respondeu à paralelização, com eficiência acima de 97% em todos os casos. O crescimento leve da eficiência com entradas maiores bate com o que a Lei de Gustafson prevê: para matrizes grandes, o overhead de gerenciar threads fica insignificante perto do trabalho útil.

5.2 Selection Sort

5.2.1 Descrição

O Selection Sort divide o vetor em duas partes: ordenada (à esquerda, começa vazia) e não-ordenada (à direita, começa com tudo). Em cada passo seleciona o menor elemento da parte não-ordenada e troca com a primeira posição não-ordenada.

Exemplo com [64, 25, 12, 22, 11]:

- Passo 1: mínimo = 11. Troca 64 $\leftrightarrow$ 11: [11, 25, 12, 22, 64]
- Passo 2: mínimo = 12. Troca 25 $\leftrightarrow$ 12: [11, 12, 25, 22, 64]
- Passo 3: mínimo = 22. Troca 25 $\leftrightarrow$ 22: [11, 12, 22, 25, 64]
- Passo 4: mínimo = 25. Sem troca: [11, 12, 22, 25, 64]

5.2.2 Complexidade

$\Theta(n^2)$ no tempo (melhor, médio e pior caso são iguais). $O(1)$ de espaço auxiliar.

5.2.3 Implementação Serial

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4
5 int main(int argc, char *argv[]) {
6     if (argc < 3) {
7         fprintf(stderr, "Uso: %s <tamanho> <arquivo>\n", argv[0]);
8         return 1;
9     }
10    int n = atoi(argv[1]);
11    float *v = (float *)malloc(n * sizeof(float));
12
13    FILE *f = fopen(argv[2], "rb");
14    fread(v, sizeof(float), n, f);
15    fclose(f);
16
17    double t_inicio = omp_get_wtime();
18
19    for (int i = 0; i < n - 1; i++) {
20        int min_idx = i;
21        for (int j = i + 1; j < n; j++) {
22            if (v[j] < v[min_idx])
23                min_idx = j;
24        }
25        if (min_idx != i) {
26            float tmp = v[i];
27            v[i] = v[min_idx];
28            v[min_idx] = tmp;
29        }
30    }
31
32    double t_fim = omp_get_wtime();
33    printf("Tempo serial: %.6f s\n", t_fim - t_inicio);
34
35    FILE *out = fopen("vetor_ordenado.out", "wb");
36    fwrite(v, sizeof(float), n, out);
37    fclose(out);
38    free(v);
39    return 0;
40 }
```

Listing 3: Selection Sort serial

5.2.4 Dependências de Dados

O laço externo (i) tem uma dependência forte: na iteração i o vetor é modificado pela troca, e a iteração $i + 1$ lê o vetor modificado. Não dá pra paralelizar diretamente.

O laço interno (j) é independente entre iterações, mas a atualização de `min_idx` cria uma condição de corrida se duas threads tentarem atualizar ao mesmo tempo. A solução é cada thread manter seu próprio mínimo local e depois combinar com uma região crítica.

5.2.5 Implementação Paralela

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4
5 int main(int argc, char *argv[]) {
6     if (argc < 3) {
7         fprintf(stderr, "Uso: %s <tamanho> <arquivo>\n", argv[0]);
8         return 1;
9     }
10    int n = atoi(argv[1]);
11    float *v = (float *)malloc(n * sizeof(float));
12
13    FILE *f = fopen(argv[2], "rb");
14    fread(v, sizeof(float), n, f);
15    fclose(f);
16
17    double t_inicio = omp_get_wtime();
18
19    for (int i = 0; i < n - 1; i++) {
20        int min_idx = i;
21        float min_val = v[i];
22
23        #pragma omp parallel
24        {
25            int local_min_idx = i;
26            float local_min_val = v[i];
27
28            #pragma omp for nowait
29            for (int j = i + 1; j < n; j++) {
30                if (v[j] < local_min_val) {
31                    local_min_val = v[j];
32                    local_min_idx = j;
33                }
34            }
35
36            /* cada thread atualiza o minimo global uma de cada vez */
37            #pragma omp critical
38            {
39                if (local_min_val < min_val) {
40                    min_val = local_min_val;
41                    min_idx = local_min_idx;
42                }
43            }
44        }
45
46        if (min_idx != i) {
47            float tmp = v[i];
48            v[i] = v[min_idx];
49            v[min_idx] = tmp;
50        }
51    }
52
53    double t_fim = omp_get_wtime();
54    printf("Tempo paralelo (%d threads): %.6f s\n",
55        omp_get_max_threads(), t_fim - t_inicio);
56}
```

```

57 FILE *out = fopen("vetor_ordenado.out", "wb");
58 fwrite(v, sizeof(float), n, out);
59 fclose(out);
60 free(v);
61 return 0;
62 }

```

Listing 4: Selection Sort paralelo

O `nowait` permite que as threads avancem direto pra região crítica sem esperar que todas terminem o laço. A região crítica em si é rápida (só uma comparação e atribuição).

5.2.6 Resultados

Table 5: Tempo de execução (s) — Selection Sort

Tamanho	1 thread	2 threads	4 threads
33.000	2,413	1,508	1,063
66.000	9,650	5,744	4,124
106.000	24,892	14,472	10,372

Table 6: Speedup — Selection Sort

Tamanho	2 threads	4 threads
33.000	1,60	2,27
66.000	1,68	2,34
106.000	1,72	2,40

Table 7: Eficiência — Selection Sort

Tamanho	2 threads	4 threads
33.000	0,801	0,569
66.000	0,840	0,584
106.000	0,860	0,600

O speedup de $2,40\times$ com 4 threads ficou bem longe do ideal. O principal culpado é o laço externo que permanece serial: para cada uma das $n - 1$ iterações a gente cria e destrói um time de threads, e esse overhead se acumula bastante. Nos testes iniciais o resultado com entradas pequenas chegou a ficar pior que o serial por causa disso.

5.3 Heap Sort

5.3.1 Descrição

O Heap Sort usa a estrutura de max-heap, onde cada nó é maior ou igual aos seus filhos. O nó na posição i tem filhos em $2i + 1$ e $2i + 2$.

O algoritmo tem duas fases:

- **Fase 1 — Construção do heap (heapify):** reestrutura o vetor para satisfazer a propriedade de max-heap, de baixo pra cima. $O(n)$ no total.
- **Fase 2 — Extração:** troca o maior elemento (raiz) com o último, reduz o heap em 1 e restaura a propriedade com sift-down. Repete $n - 1$ vezes, cada vez com custo $O(\log n)$. Total $O(n \log n)$.

5.3.2 Complexidade

$O(n \log n)$ no melhor, médio e pior caso. $O(1)$ de espaço auxiliar.

5.3.3 Implementação Serial

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4
5 void sift_down(float *v, int i, int sz) {
6     while (1) {
7         int maior = i;
8         int esq = 2 * i + 1;
9         int dir = 2 * i + 2;
10        if (esq < sz && v[esq] > v[maior]) maior = esq;
11        if (dir < sz && v[dir] > v[maior]) maior = dir;
12        if (maior == i) break;
13        float tmp = v[i]; v[i] = v[maior]; v[maior] = tmp;
14        i = maior;
15    }
16 }
17
18 int main(int argc, char *argv[]) {
19     if (argc < 3) {
20         fprintf(stderr, "Uso: %s <tamanho> <arquivo>\n", argv[0]);
21         return 1;
22     }
23     int n = atoi(argv[1]);
24     float *v = (float *)malloc(n * sizeof(float));
25
26     FILE *f = fopen(argv[2], "rb");
27     fread(v, sizeof(float), n, f);
28     fclose(f);
29
30     double t_inicio = omp_get_wtime();
31
32     /* fase 1: constroi o heap de baixo pra cima */
33     for (int i = n / 2 - 1; i >= 0; i--)
34         sift_down(v, i, n);
35
36     /* fase 2: extrai o maximo repetidamente -- serial por dependencia */
37     for (int i = n - 1; i > 0; i--) {
38         float tmp = v[0]; v[0] = v[i]; v[i] = tmp;
39         sift_down(v, 0, i);
40     }
41
42     double t_fim = omp_get_wtime();

```

```

43     printf("Tempo serial: %.6f s\n", t_fim - t_inicio);
44
45     FILE *out = fopen("vetor_ordenado.out", "wb");
46     fwrite(v, sizeof(float), n, out);
47     fclose(out);
48     free(v);
49     return 0;
50 }

```

Listing 5: Heap Sort serial

5.3.4 Dependências de Dados

A fase de construção tem uma dependência entre níveis da árvore: o sift-down de um nó pai depende do estado dos filhos. Mas nós do mesmo nível são independentes entre si, então dá pra paralelizar dentro de cada nível.

A fase de extração é serial sem saída. Cada iteração depende do que a anterior fez no heap. Não tem jeito de fazer duas extrações ao mesmo tempo.

5.3.5 Implementação Paralela

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4
5  void sift_down(float *v, int i, int sz) {
6      while (1) {
7          int maior = i;
8          int esq = 2 * i + 1;
9          int dir = 2 * i + 2;
10         if (esq < sz && v[esq] > v[maior]) maior = esq;
11         if (dir < sz && v[dir] > v[maior]) maior = dir;
12         if (maior == i) break;
13         float tmp = v[i]; v[i] = v[maior]; v[maior] = tmp;
14         i = maior;
15     }
16 }
17
18 int main(int argc, char *argv[]) {
19     if (argc < 3) {
20         fprintf(stderr, "Uso: %s <tamanho> <arquivo>\n", argv[0]);
21         return 1;
22     }
23     int n = atoi(argv[1]);
24     float *v = (float *)malloc(n * sizeof(float));
25
26     FILE *f = fopen(argv[2], "rb");
27     fread(v, sizeof(float), n, f);
28     fclose(f);
29
30     double t_inicio = omp_get_wtime();
31
32     /* fase 1 paralela: processa nivel a nivel com barreira entre eles */
33     int nivel_inicio = n / 2 - 1;
34     int nivel_atual = nivel_inicio;

```

```

35     int nivel_fim;
36
37     while (nivel_atual >= 0) {
38         nivel_fim = nivel_atual;
39         if (nivel_atual > 0)
40             nivel_atual = nivel_atual / 2 - 1
41                         + (nivel_atual % 2 == 0 ? -1 : 0);
42
43         /* nos do mesmo nivel sao independentes */
44         #pragma omp parallel for schedule(static)
45         for (int i = nivel_fim; i >= nivel_atual + 1; i--)
46             sift_down(v, i, n);
47
48         if (nivel_atual < 0) break;
49         nivel_atual--;
50     }
51
52     sift_down(v, 0, n);
53
54     /* fase 2: serial -- nao tem como paralelizar */
55     for (int i = n - 1; i > 0; i--) {
56         float tmp = v[0]; v[0] = v[i]; v[i] = tmp;
57         sift_down(v, 0, i);
58     }
59
60     double t_fim = omp_get_wtime();
61     printf("Tempo paralelo (%d threads): %.6f s\n",
62           omp_get_max_threads(), t_fim - t_inicio);
63
64     FILE *out = fopen("vetor_ordenado.out", "wb");
65     fwrite(v, sizeof(float), n, out);
66     fclose(out);
67     free(v);
68     return 0;
69 }

```

Listing 6: Heap Sort paralelo

5.3.6 Resultados

Table 8: Tempo de execução (s) — Heap Sort

Tamanho	1 thread	2 threads	4 threads
20.000.000	6,133	4,946	4,381
40.000.000	12,771	9,824	8,240
80.000.000	26,554	19,525	15,439

Table 9: Speedup — Heap Sort

Tamanho	2 threads	4 threads
20.000.000	1,24	1,40
40.000.000	1,30	1,55
80.000.000	1,36	1,72

Table 10: Eficiência — Heap Sort

Tamanho	2 threads	4 threads
20.000.000	0,620	0,350
40.000.000	0,648	0,387
80.000.000	0,679	0,429

Com 4 threads a eficiência não passou de 0,43. A fase de extração, que domina o tempo total por ser $O(n \log n)$ contra $O(n)$ da construção, ficou 100% serial. É um caso claro do que a Lei de Amdahl prevê: uma fração grande serial limita muito o speedup.

5.4 Histograma

5.4.1 Descrição

O histograma conta quantas vezes cada valor aparece num vetor. Formalmente:

$$H[v] = |\{i : V[i] = v\}|, \quad \forall v \in \{\min(V), \dots, \max(V)\} \quad (6)$$

O algoritmo percorre o vetor uma vez e incrementa o contador de cada valor encontrado. Simples, mas com um problema sério na versão paralela.

5.4.2 Complexidade

$O(n)$ no tempo, $O(r)$ de espaço, onde $r = \max(V) - \min(V) + 1$.

5.4.3 Implementação Serial

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <limits.h>
4 #include <omp.h>
5
6 int main(void) {
7     FILE *f = fopen("vetor.in", "rb");
8     if (!f) { perror("fopen"); return 1; }
9
10    fseek(f, 0, SEEK_END);
11    long sz = ftell(f);
12    rewind(f);
13    int n = (int)(sz / sizeof(int));
14
15    int *v = (int *)malloc(n * sizeof(int));

```

```

16 fread(v, sizeof(int), n, f);
17 fclose(f);
18
19 int vmin = INT_MAX, vmax = INT_MIN;
20 for (int i = 0; i < n; i++) {
21     if (v[i] < vmin) vmin = v[i];
22     if (v[i] > vmax) vmax = v[i];
23 }
24
25 int range = vmax - vmin + 1;
26 int *hist = (int *)calloc(range, sizeof(int));
27
28 double t_inicio = omp_get_wtime();
29
30 for (int i = 0; i < n; i++)
31     hist[v[i] - vmin]++;
32
33 double t_fim = omp_get_wtime();
34 printf("Tempo serial: %.6f s\n", t_fim - t_inicio);
35
36 FILE *out = fopen("histograma.out", "w");
37 for (int i = 0; i < range; i++)
38     if (hist[i] > 0)
39         fprintf(out, "%d %d\n", i + vmin, hist[i]);
40 fclose(out);
41
42 free(v); free(hist);
43 return 0;
44 }

```

Listing 7: Histograma serial

5.4.4 Condição de Corrida

O `hist[v[i] - vmin]++` é uma operação de leitura-modificação-escrita. Se duas threads tentarem incrementar a mesma posição ao mesmo tempo, as duas leem o mesmo valor, as duas somam 1, e as duas escrevem o mesmo resultado — perdendo um incremento.

Três opções pra resolver isso no OpenMP:

1. Região crítica global: serializa tudo, joga fora o ganho do paralelismo.
2. Operação atômica: usa instrução atômica do hardware, melhor que crítica mas ainda gera contenção com muitos conflitos.
3. Histogramas privados por thread: cada thread tem o próprio histograma e no final somam tudo. Sem conflito na fase de contagem.

Usamos a terceira opção.

5.4.5 Implementação Paralela

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <limits.h>
5 #include <omp.h>

```

```

6
7 int main(void) {
8     FILE *f = fopen("vetor.in", "rb");
9     if (!f) { perror("fopen"); return 1; }
10
11     fseek(f, 0, SEEK_END);
12     long sz = ftell(f);
13     rewind(f);
14     int n = (int)(sz / sizeof(int));
15
16     int *v = (int *)malloc(n * sizeof(int));
17     fread(v, sizeof(int), n, f);
18     fclose(f);
19
20     int vmin = INT_MAX, vmax = INT_MIN;
21     #pragma omp parallel for reduction(min:vmin) reduction(max:vmax)
22     for (int i = 0; i < n; i++) {
23         if (v[i] < vmin) vmin = v[i];
24         if (v[i] > vmax) vmax = v[i];
25     }
26
27     int range = vmax - vmin + 1;
28     int *hist = (int *)calloc(range, sizeof(int));
29     int p = omp_get_max_threads();
30
31     /* uma copia do histograma por thread */
32     int **hist_priv = (int **)malloc(p * sizeof(int *));
33     for (int t = 0; t < p; t++)
34         hist_priv[t] = (int *)calloc(range, sizeof(int));
35
36     double t_inicio = omp_get_wtime();
37
38     #pragma omp parallel
39     {
40         int tid = omp_get_thread_num();
41         int *h_local = hist_priv[tid];
42
43         #pragma omp for schedule(static)
44         for (int i = 0; i < n; i++)
45             h_local[v[i] - vmin]++; /* sem conflito */
46
47         /* reducao: soma os histogramas privados */
48         #pragma omp for schedule(static)
49         for (int j = 0; j < range; j++) {
50             int soma = 0;
51             for (int t = 0; t < p; t++)
52                 soma += hist_priv[t][j];
53             hist[j] = soma;
54         }
55     }
56
57     double t_fim = omp_get_wtime();
58     printf("Tempo paralelo (%d threads): %.6f s\n", p, t_fim - t_inicio);
59
60     FILE *out = fopen("histograma.out", "w");
61     for (int j = 0; j < range; j++)
62         if (hist[j] > 0)

```

```

63         fprintf(out, "%d %d\n", j + vmin, hist[j]);
64     fclose(out);
65
66     for (int t = 0; t < p; t++) free(hist_priv[t]);
67     free(hist_priv); free(v); free(hist);
68     return 0;
69 }

```

Listing 8: Histograma paralelo com histogramas privados

A barreira implícita no final do primeiro `for` garante que todos os histogramas privados estão prontos antes de começar a redução. O segundo `for` também paralleliza a própria redução.

5.4.6 Resultados

Table 11: Tempo de execução (s) — Histograma

Tamanho	1 thread	2 threads	4 threads
150.000.000	6,6000	3,4555	1,9527
300.000.000	13,2000	6,7347	3,5967
600.000.000	26,4000	13,4010	7,2329

Table 12: Speedup — Histograma

Tamanho	2 threads	4 threads
150.000.000	1,91	3,38
300.000.000	1,96	3,67
600.000.000	1,97	3,65

Table 13: Eficiência — Histograma

Tamanho	2 threads	4 threads
150.000.000	0,955	0,845
300.000.000	0,980	0,917
600.000.000	0,985	0,912

O histograma escalou muito bem. A pequena queda de eficiência com 4 threads vem da fase de redução, que acessa a memória de todas as cópias privadas e gera mais pressão no cache. Mesmo assim ficou acima de 91% para as entradas maiores.

5.5 Makefile

```

1 CC = gcc
2 CFLAGS = -O2 -fopenmp -Wall
3 LIBPPC_INC = -I./LibPPC/include
4 LIBPPC_SRC = ./LibPPC/src/libppc.c

```

```

5 BINS = matrixmult_serial matrixmult_paralelo \
6     selection_serial selection_paralelo \
7     heapsort_serial heapsort_paralelo \
8     histograma_serial histograma_paralelo
9
10 all: $(BINS)
11
12 matrixmult_serial: matrixmult_serial.c
13     $(CC) $(CFLAGS) -o $@ $<
14
15 matrixmult_paralelo: matrixmult_paralelo.c
16     $(CC) $(CFLAGS) -o $@ $<
17
18 selection_serial: selection_serial.c
19     $(CC) $(CFLAGS) -o $@ $<
20
21 selection_paralelo: selection_paralelo.c
22     $(CC) $(CFLAGS) -o $@ $<
23
24 heapsort_serial: heapsort_serial.c
25     $(CC) $(CFLAGS) -o $@ $<
26
27 heapsort_paralelo: heapsort_paralelo.c
28     $(CC) $(CFLAGS) -o $@ $<
29
30 histograma_serial: histograma_serial.c
31     $(CC) $(CFLAGS) -o $@ $<
32
33 histograma_paralelo: histograma_paralelo.c
34     $(CC) $(CFLAGS) -o $@ $<
35
36 clean:
37     rm -f $(BINS) *.out *.in
38
39 .PHONY: all clean

```

Listing 9: Makefile

6 Resultados e Discussão

6.1 Visão Geral do Speedup

O gráfico abaixo resume o speedup de cada algoritmo para o maior tamanho de entrada:

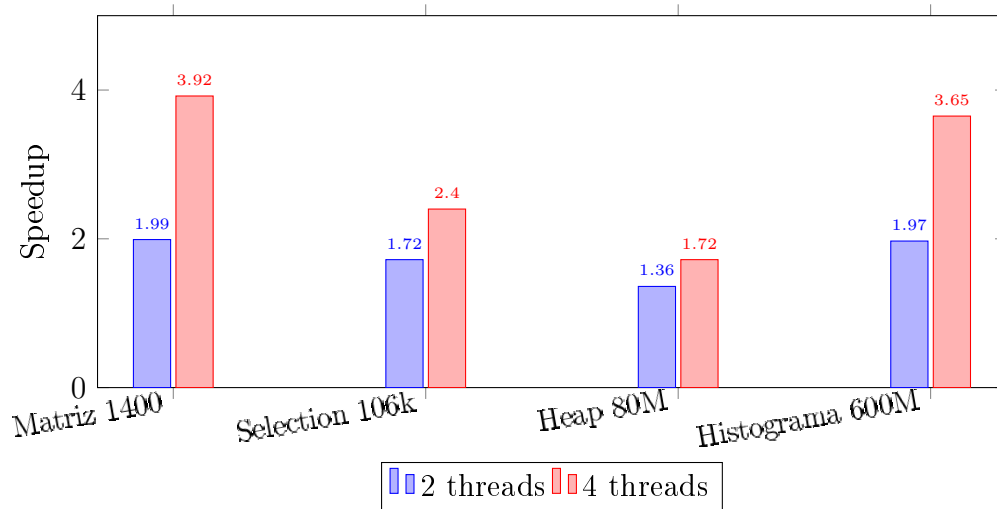


Figure 1: Speedup por algoritmo para 2 e 4 threads (maior tamanho de entrada)

A diferença entre os grupos é bem clara: multiplicação matricial e histograma ficam perto do speedup ideal, enquanto os sorts ficam bem abaixo. Isso reflete diretamente a estrutura de dependências de cada algoritmo.

6.2 Speedup em Função do Tamanho

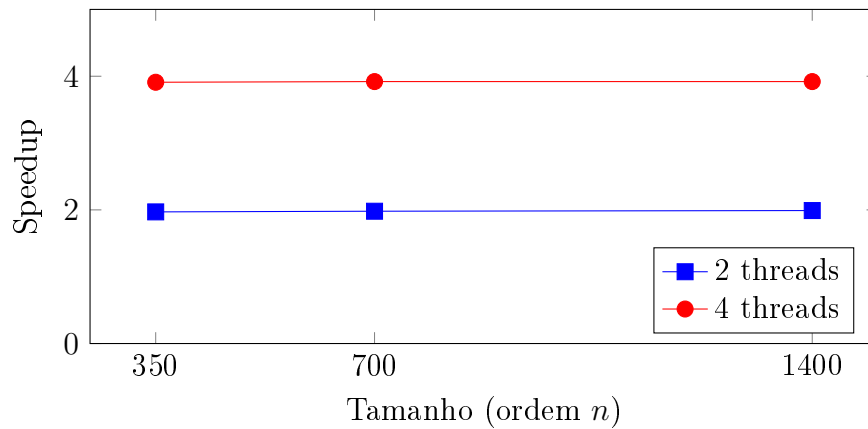


Figure 2: Speedup vs. tamanho — Multiplicação Matricial

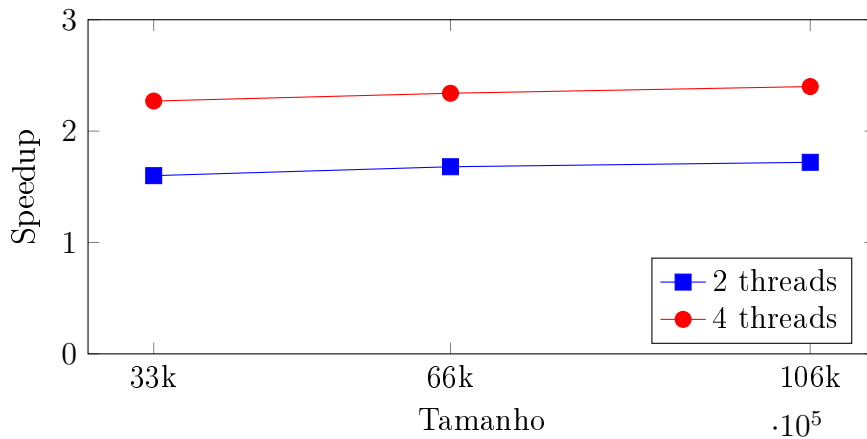


Figure 3: Speedup vs. tamanho — Selection Sort

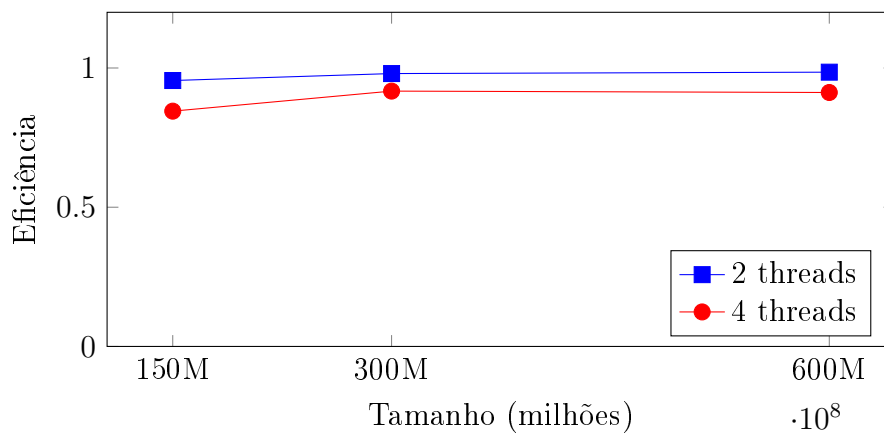


Figure 4: Eficiência vs. tamanho — Histograma

Em todos os casos o speedup cresce (ou pelo menos se mantém) com entradas maiores. Isso é o efeito que a Lei de Gustafson descreve: problemas maiores diluem o overhead.

6.3 Comparação Selection Sort vs Heap Sort

6.3.1 Qual gera mais cache-misses?

O Heap Sort. O problema é o padrão de acesso nas operações de sift-down: o caminho da raiz até as folhas corresponde a índices que dobram a cada passo ($i \rightarrow 2i + 1, 2i + 2$). Para heaps grandes os elementos acessados estão espalhados por toda a memória, gerando cache-misses frequentes. LaMarca e Ladner [12] mostraram empiricamente que o Heap Sort tem muito mais cache-misses que o Merge Sort ou o Quick Sort para entradas que não cabem no cache.

O Selection Sort acessa o vetor de forma sequencial no laço interno (boa localidade espacial), mas a troca final pega um elemento de posição aleatória. O impacto é menor que o do Heap Sort, especialmente para entradas moderadas.

6.3.2 Qual se beneficiou mais do paralelismo?

O Selection Sort, com speedup de $2,40\times$ contra $1,72\times$ do Heap Sort com 4 threads. Parece contra-intuitivo já que o Heap Sort é assintoticamente mais eficiente. A explicação

está na fração serial de cada um.

No Selection Sort a fase paralelizável (busca do mínimo) representa uma fatia razoável do tempo total porque o algoritmo é $O(n^2)$ e essa busca custa $O(n)$ por iteração. No Heap Sort a fase de extração — completamente serial — domina o tempo porque é $O(n \log n)$ contra $O(n)$ da construção do heap. Sobra muito pouco pra paralelizar.

6.3.3 Balanceamento de carga

No Selection Sort paralelo, o laço interno na iteração i tem $n - i - 1$ elementos. Nas primeiras iterações o trabalho é grande e as threads ficam bem ocupadas. Nas últimas iterações o laço tem poucos elementos e o overhead de criar o time de threads supera o trabalho útil. É um desbalanceamento que piora gradualmente.

No Heap Sort, a construção por níveis tem o problema oposto nos níveis superiores da árvore: os primeiros níveis têm poucos nós (o nível da raiz tem 1 nó), então praticamente uma thread faz todo o trabalho. Para esses casos o overhead de gerenciar 4 threads pra processar 1 ou 2 elementos é indefensável.

No geral o Selection Sort tem balanceamento melhor que o Heap Sort por causa das iterações do laço interno terem custo uniforme (uma comparação cada) — a divisão estática os divide igualmente.

6.4 Análise pela Lei de Amdahl

Usando os resultados com 4 threads para estimar a fração serial de cada algoritmo:

$$f = \frac{1/S - 1/p}{1 - 1/p} = \frac{1/S - 1/4}{3/4} \quad (7)$$

Table 14: Fração serial estimada via Lei de Amdahl

Algoritmo	S(4)	Fração serial estimada
Multiplicação Matricial	3,92	0,7%
Selection Sort	2,40	22,2%
Heap Sort	1,72	44,2%
Histograma	3,65	3,2%

Os números batem com o que a análise qualitativa do código sugeria. A fração serial de 44,2% do Heap Sort explica diretamente por que ele escala tão mal, mesmo tendo complexidade assintótica melhor que o Selection Sort.

7 Limitações

Os experimentos rodaram num ambiente Windows 10 de propósito geral, com outros processos ativos em segundo plano. Isso cria variabilidade nos tempos. A mediana de 5 execuções ajuda, mas um ambiente controlado (sem outros processos, com afinidade de threads fixada) daria resultados mais limpos.

Testamos no máximo 4 threads, o que representa menos da metade dos núcleos do Ryzen 5 3500X (6 núcleos). Seria interessante testar com 6 threads e verificar se a eficiência

se mantém ou cai. Com mais threads o overhead de sincronização tende a crescer e pode começar a dominar, especialmente nos sorts.

A implementação do Heap Sort paralelo por níveis é didática mas não é a mais eficiente. Usar `#pragma omp task` com divisão recursiva do heap provavelmente daria speedups melhores.

8 Considerações Finais

Esse trabalho mostrou na prática o que a teoria de paralelismo prevê: nem todo algoritmo se beneficia igualmente de mais threads. A multiplicação matricial e o histograma escalaram perto do ideal; os sorts não.

O ponto central é que paralelizabilidade depende de dependências de dados. Onde as iterações são independentes (multiplicação matricial, contagem do histograma) o ganho é quase linear. Onde há uma cadeia de dependências que precisa ser respeitada (laço externo do Selection Sort, fase de extração do Heap Sort) o ganho é limitado.

A comparação entre as três estratégias pro histograma (região crítica, atômico, histogramas privados) ilustra bem como a escolha da estratégia de sincronização importa tanto quanto decidir paralelizar. A versão com histogramas privados eliminou completamente a contenção durante a contagem e ficou com eficiência acima de 90%.

A comparação entre Selection Sort e Heap Sort foi talvez o resultado mais surpreendente: o algoritmo assintoticamente inferior ($O(n^2)$) se beneficiou mais do paralelismo que o superior ($O(n \log n)$) por ter uma fração serial menor na prática.

Como próximos passos ficam: algoritmos de ordenação intrinsecamente paralelos (Merge Sort, Bitonic Sort), técnicas de blocking na multiplicação matricial pra melhorar o uso de cache em conjunto com o paralelismo, e uma comparação entre OpenMP e MPI pra os mesmos algoritmos.

References

- [1] AMDAHL, G. M. *Validity of the single processor approach to achieving large scale computing capabilities*. In: Proceedings of the Spring Joint Computer Conference (AFIPS). New York: ACM, 1967. p. 483–485.
- [2] GUSTAFSON, J. L. *Reevaluating Amdahl's law*. Communications of the ACM, New York, v. 31, n. 5, p. 532–533, May 1988.
- [3] OPENMP ARCHITECTURE REVIEW BOARD. *OpenMP Application Programming Interface Specification Version 5.2*. [S.l.]: OpenMP ARB, 2021. Disponível em: <https://www.openmp.org/specifications/>.
- [4] FLYNN, M. J. *Some computer organizations and their effectiveness*. IEEE Transactions on Computers, New York, v. C-21, n. 9, p. 948–960, Sep. 1972.
- [5] DAGUM, L.; MENON, R. *OpenMP: an industry-standard API for shared-memory programming*. IEEE Computational Science and Engineering, New York, v. 5, n. 1, p. 46–55, Jan./Mar. 1998.
- [6] CHAPMAN, B.; JOST, G.; VAN DER PAS, R. *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge: MIT Press, 2007.

- [7] GOTO, K.; VAN DE GEIJN, R. A. *Anatomy of high-performance matrix multiplication*. ACM Transactions on Mathematical Software, New York, v. 34, n. 3, p. 12:1–12:25, May 2008.
- [8] ANDERSON, E. et al. *LAPACK Users' Guide*. 3. ed. Philadelphia: SIAM, 1999.
- [9] STRASSEN, V. *Gaussian elimination is not optimal*. Numerische Mathematik, Berlin, v. 13, p. 354–356, 1969.
- [10] SINGLER, J.; SANDERS, P.; PUTZE, F. *MCSTL: The multi-core standard template library*. In: European Conference on Parallel Processing (Euro-Par 2007). Lecture Notes in Computer Science, v. 4641. Berlin: Springer, 2007. p. 682–694.
- [11] NAKASHIMA, H. et al. *Performance evaluation of OpenMP and MPI on a shared-memory multiprocessor*. Parallel Computing, Amsterdam, v. 38, n. 1–2, p. 1–14, Jan. 2012.
- [12] LAMARCA, A.; LADNER, R. E. *The influence of caches on the performance of sorting*. Journal of Algorithms, Amsterdam, v. 31, n. 1, p. 66–104, Apr. 1999.
- [13] CORMEN, T. H. et al. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009.
- [14] PACHECO, P. *An Introduction to Parallel Programming*. Burlington: Morgan Kaufmann, 2011.
- [15] QUINN, M. J. *Parallel Programming in C with MPI and OpenMP*. New York: McGraw-Hill, 2004.
- [16] HAGER, G.; WELLEIN, G. *Introduction to High Performance Computing for Scientists and Engineers*. Boca Raton: CRC Press, 2011.